

Development Log for Software Design

Project Title: Waveform analyser

Date Started: 26th March 2026

Development Period: 5 Weeks

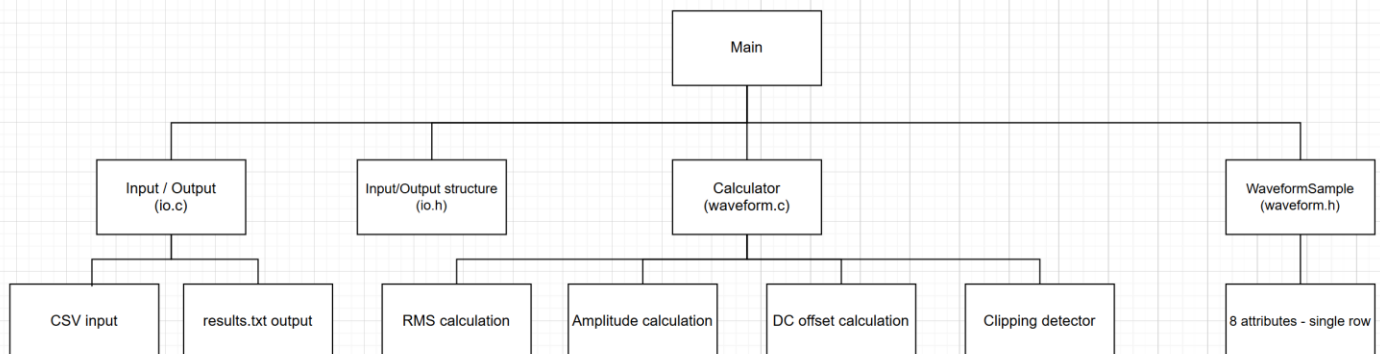
Author: Jason Chen

Student ID: 24015280

Github repository - [jas0npanda/waveform-analyser-tb2: Mini coursework for programming module in tb2 \(UWE\)](#)

1. Structure Chart

A structure chart is created to visually represent the different modules in the system. Below is a simple structure chart of the system.



2. Program Design (Pseudocode/Flowcharts)

Psuedocode for io.c – function – load_csv()

Version 1.0

Requirements:

4	File I/O to load data	Open the CSV, parse the header row, read each data row into your struct array, and handle file-not-found errors gracefully
---	-----------------------	--

Int load_csv(WaveformSample)

FILE *fp (pointer) = fopen('power_quality_log.csv', r)

if fp = null

 fprintf('Error: cannot open file')

 return 1

count_rows(file) (Returns rows)

if rows = null

 fprintf("Error, file has no data rows")

 return 2

rows = rows – 1 (Taking header row into account)

WaveformSample *p = arr (ptr)

while rows > 0

 int n = scanf('power_quality_log.csv'),

 ptr = timestamp

 ptr = phase_A_voltage

 ptr = phase_B_voltage

 ptr = phase_C_voltage

 ptr = line_current

 ptr = frequency

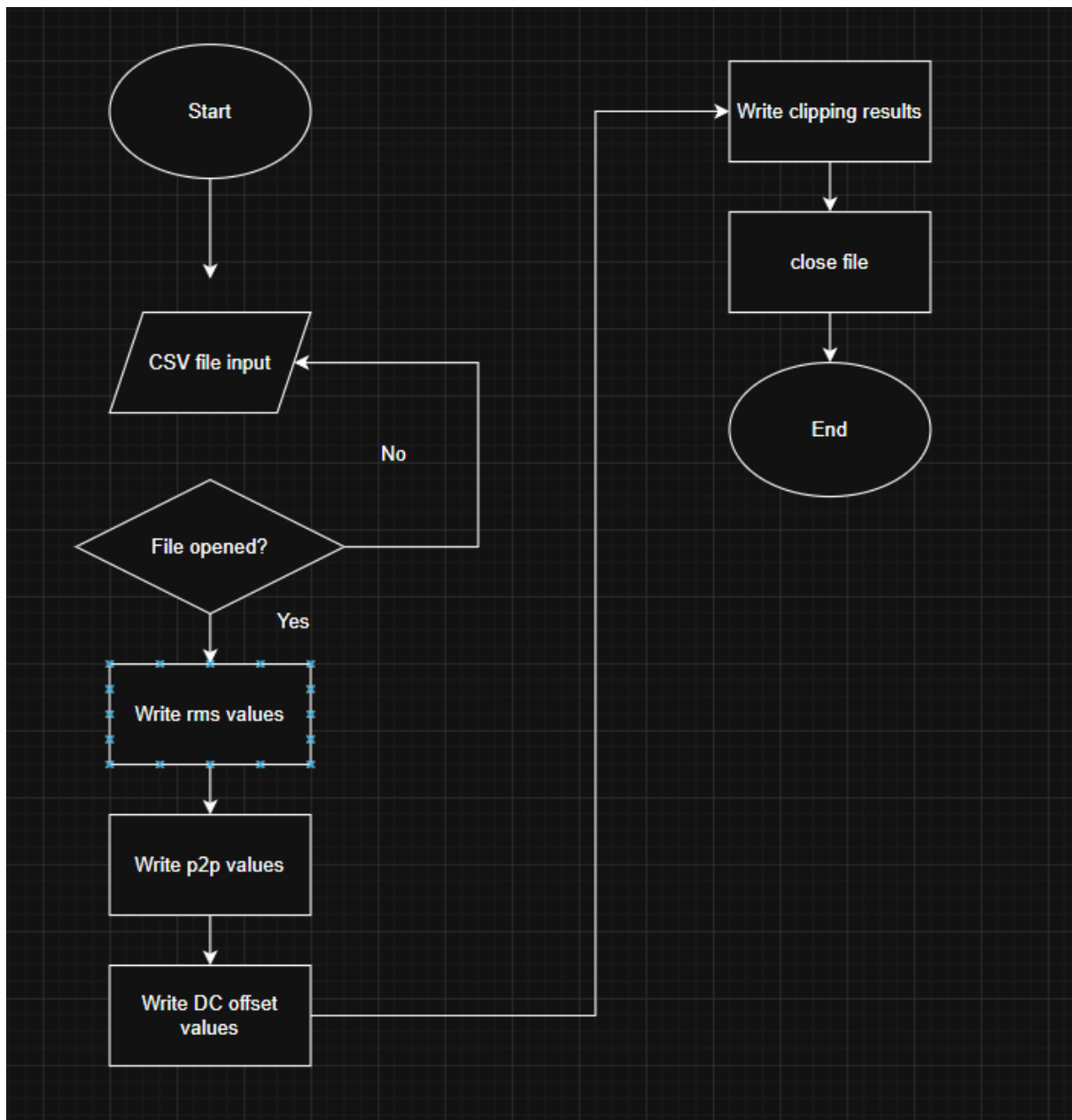
 ptr = power_factor

 ptr = thd_percent

fclose('power_quality_log.csv')

return 0

Flowchart for io.c, write_results



3. Function Description

Each function in the system is described below:

- Load_csv, version 2.0
 - Description:
 - The function reads a csv file (waveform data), dynamically allocating memory for each row, using the waveform sample structure
 - Input Parameters:
 - char – filename (csv file)
 - int number_of_rows – pointer to store number of rows read
 - Returns:
 - NULL – file cannot be opened or the malloc call has failed
 - Waveform sample array location index
- Compute_rms, version 1.0
 - Description
 - Calculates rms of the dataset
 - Input parameters
 - double – array of voltage values
 - Returns
 - Double – RMS value
- Compute_peaktopeak, version 1.0
 - Description
 - Calculates max – min value for input AC voltage dataset
 - Input parameters
 - Double – array of voltage values
 - Returns
 - Double – peak to peak value

- Compute_mean, version 1.0
 - Description
 - Calculates mean value for dataset (DC offset)
 - Input parameters
 - Double – array of voltage values
 - Int count (counter)
 - Returns
 - Double – mean value
- detect_clipping, version 1.0
 - Description
 - See if any values exceed clipping threshold (324.9V)
 - Input parameters
 - Double – array of voltage values
 - Int count (counter)
 - Double threshold
 - Returns
 - 1 – Clipping detected
 - 0 – Clipping not detected
- Check_tolerance, version 1.0
 - Description
 - Check each phases' RMS value to see if they are within 10% of the nominal value (230V)
 - Input parameters
 - Double – rms value
 - Double nominal
 - Returns
 - 1 – Within tolerance
 - 0 – Not within tolerance

- detect_clipping, version 1.0
 - Description
 - See if any values exceed clipping threshold (324.9V)
 - Input parameters
 - Double – array of voltage values
 - Int count (counter)
 - Double threshold
 - Returns
 - 1 – Clipping detected
 - 0 – Clipping not detected
- Detect_clipping, version 2.0
 - Description
 - See if any values exceed clipping threshold
 - Count how many values have clipped
 - Input parameters
 - Double – voltage values array
 - Int count – counter
 - Double threshold
 - Returns
 - Clipped_values – number of clipped values

- Write_results, version 1.0
 - Description
 - Outputs a txt file of the waveform analysis results
 - Input parameters
 - Char - filename
 - Double rms_a, rms_b, rms_c (Phase rms voltages)
 - Double p2p_a, p2p_b, p2p_c (Peak to peak)
 - Double dc_a, dc_b, dc_c (DC offset)
 - int clip_a, clip_b, clip_c (Clipping flag)
 - Returns
 - Void – no value returned
 - Text file – results

Design rationale

```
struct WaveformSample {  
    double timestamp;  
    double phase_A_voltage;  
    double phase_B_voltage;  
    double phase_C_voltage;  
    double line_current;  
    double frequency;  
    double power_factor;  
    double thd_percent;  
};
```

I included all entries from csv into my struct for flexibility in working with my program. I did not tackle the extension challenges but including extra entries, such as power_factor, would allow me to extend my analyser further.

I also used double in order to increase accuracy and avoid rounding errors. I could see from the CSV file that each file was extremely long in numbers, so this was another reason behind using the double format.

Pointer / memory management

```
struct WaveformSample* samples = malloc(count * sizeof(struct  
WaveformSample));  
if (!samples) return NULL;
```

My malloc call in io.c here calculates the size of each row of my struct – WaveformSample (64 bytes), then multiplies this by count (one sample * total number of rows) to find out exactly how much memory I need for this data set.

Malloc() then asks the operating system to allocate this much memory for me and if it fails, it will return NULL (next line)

This is then stored into 'samples' treating it as one long array that I can index further down to extract the data I need.

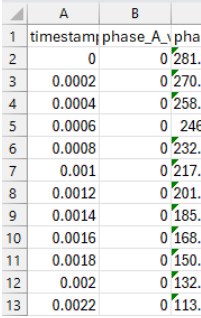
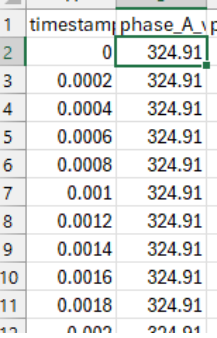
The way I have used pointers in my program is similar to array indexing, like in io.c, I utilise;

```
while (fgets(line, max_line, fp)) {  
    sscanf(line, "%lf,%lf,%lf,%lf,%lf,%lf,%lf,%lf",  
        &samples[i].timestamp,  
        &samples[i].phase_A_voltage,  
        &samples[i].phase_B_voltage,  
        &samples[i].phase_C_voltage,  
        &samples[i].line_current,  
        &samples[i].frequency,  
        &samples[i].power_factor,  
        &samples[i].thd_percent);
```

Since I used WaveformSample* samples prior, I can now treat each structure as an array to find the data I need. The & symbol is key as it is the address operator, which takes the address of the data. The reasoning for this method is for readability, since you can clearly see samples[i] represents the i'th position of the row, which makes it easier to trace the logic through.

Later in my code, specifically in main.c, I used free(a), free(b), etc. Without this I would experience memory leaks, as shown in my github commits.

Testing

Input variables	Expected output	Actual output	Test outcome
Test1.csv – 0s in phase_A_voltage column 	0 for rms, peak to peak, dc offset and clipping values, 0 for tolerance value	RMS 0, P2P 0, DC 0, Tolerance 0	Pass
Test2.csv – blank csv file	No rows/data found in csv – program terminates	CSV still processed Every value is 0 (rms, etc)	Fail
Test3.csv – 324.9s in phase a column (just above threshold) 	1000 clipping values	1000 clipping values found	Pass
Test4.csv (Doesn't exist)	Null, nothing	Error: failed to load data	Pass

Test 1 – output

```
PS C:\Users\jason\CLionProjects\waveform-analyser-tb2\cmake-build-debug> .\waveform_analyser_tb2.exe test1.csv
Waveform Analyser started
Input file: test1.csv
Loading CSV...
Loaded 1000 samples
RMS A: 0.00000000, Peak-to-Peak A: 0.00000000, DC Offset A: 0.000000000000000000, Clipping values found in A: 0
RMS B: 229.80970389, Peak-to-Peak B: 649.85744423, DC Offset B: -0.00000000019849642285, Clipping values found in B: 20
RMS C: 229.80970389, Peak-to-Peak C: 649.85744423, DC Offset C: -0.00000000014214396060, Clipping values found in C: 20
RMS A within tolerance: 0
RMS B within tolerance: 1
RMS C within tolerance: 1
Power quality log analysed successfully.
Results outputted to waveform_results.txt.
```

Test 2 – output

```
PS C:\Users\jason\CLionProjects\waveform-analyser-tb2\cmake-build-debug> .\waveform_analyser_tb2.exe test2.csv
Waveform Analyser started
Input file: test2.csv
Loading CSV...
Loaded 999 samples
RMS A: 0.00000000, Peak-to-Peak A: 0.00000000, DC Offset A: 0.000000000000000000, Clipping values found in A: 0
RMS B: 0.00000000, Peak-to-Peak B: 0.00000000, DC Offset B: 0.000000000000000000, Clipping values found in B: 0
RMS C: 0.00000000, Peak-to-Peak C: 0.00000000, DC Offset C: 0.000000000000000000, Clipping values found in C: 0
RMS A within tolerance: 0
RMS B within tolerance: 0
RMS C within tolerance: 0
Power quality log analysed successfully.
Results outputted to waveform_results.txt.
```

Test 3 – output

```
PS C:\Users\jason\CLionProjects\waveform-analyser-tb2\cmake-build-debug> .\waveform_analyser_tb2.exe test3.csv
Waveform Analyser started
Input file: test3.csv
Loading CSV...
Loaded 1000 samples
RMS A: 324.91000000, Peak-to-Peak A: 0.00000000, DC Offset A: 324.90999999999752390067, Clipping values found in A: 1000
RMS B: 229.80970389, Peak-to-Peak B: 649.85744420, DC Offset B: 0.00000000000001716671, Clipping values found in B: 20
RMS C: 229.80970389, Peak-to-Peak C: 649.85744420, DC Offset C: -0.00000000000001858780, Clipping values found in C: 20
RMS A within tolerance: 0
RMS B within tolerance: 1
RMS C within tolerance: 1
Power quality log analysed successfully.
Results outputted to waveform_results.txt.
```

Test 3 – output

```
PS C:\Users\jason\CLionProjects\waveform-analyser-tb2\cmake-build-debug> .\waveform_analyser_tb2.exe test4.csv
Waveform Analyser started
Input file: test4.csv
Loading CSV...
Error: Failed to load data

PS C:\Users\jason\CLionProjects\waveform-analyser-tb2\cmake-build-debug>
```

Testing outcomes

Test 1 – 0 values

- Outcome – Pass
- Program works as expected. In the future I would like to add boundaries to filter out outlier values, like in this test.

Test 2 – Blank csv file

- Outcome – Fail
- Program should check for blank values and terminate the program
- Current version only checks if input file argument is provided

Test 3 – Boundary testing

- Outcome – Pass
- Program functioning correctly, shows clipping detector function logic is correct
- Could be extended to filter out extreme outlier values, similar to outcomes in test 1.

Test 4 – File that does not exist

- Outcome – Pass
- Program exits immediately, thus avoiding any malfunctions, like performing code on data that does not exist (random memory)

Debugging challenges

1 – 0 samples

- Symptom – When running my analyser, I kept getting zero samples of data, even though the csv file was opening and running correctly.
- I diagnosed this by adding in debug messages in my code, like **`printf("Parsed A as: %f\n", samples[i].phase_A_voltage);`** to make sure each row was being processed through
- The fix for this was to add the function; `rewind(fp)`, which restarts the position of my pointer, then skipping the header line again before proceeding with the rest of my code.
 - This worked because my issue lied in the fact my code was indexing through the dataset normally but starting at the bottom of the dataset (position 1000), therefore I was getting 0 samples loaded every time.

2 – Testing phase

- As shown earlier, I was able to have a running environment where I was able to compile my analyser, and choose which file I wanted to run, such as `test1.csv`, `test2.csv`.
- I ran into this issue, where no matter what file I chose, as long as there was an argument, the program would run and output my results fine, and it was the same results too every time as well.
- I was able to fix this by scanning through my code and eventually finding that I hardcoded into my `load_csv` function to open 'power_quality_log.csv', rather than the given filename, which I defined earlier in the program.